

MENUMIND: Autonomous Cafe Operations Agent

A Zero-Code Operations Engine Built for Small Food Businesses

Platform: #AISeekho 2026 Phase II | Submission Date: May 2026 | Build Timeline: 72 Hours

1. Executive Summary

In today's fast-paced food and beverage industry, small eateries and cafes operate on exceptionally thin margins. Managers are constantly flooded with disorganized, unstructured operational signals, ranging from informal supplier WhatsApp messages ("chicken short out of stock today") to volatile weather alerts and competitor pricing shifts. Standard AI tools fail here because they stop at summarization or passive analysis.

MenuMind is an autonomous operational agent designed to protect small businesses from supply shocks, demand surges, and pricing volatility. Built entirely over a 72-hour period using free visual tools, MenuMind reads unstructured inputs, extracts core business insights, reasons through the direct operational impact, and executes real-time system state changes. It automatically hides unavailable items, ethically adjusts menu prices, updates the live backend database, and alerts kitchen staff—all without requiring a single line of custom code.

2. Alignment with Challenge 1 Requirements

MenuMind strictly fulfills all evaluation parameters laid out in the official **Challenge 1: Autonomous Content-to-Action Agent (Insight → Action System)** specification:

- **Content Understanding:** Ingests messy, unstructured text formats, including multi-lingual Roman Urdu/English supplier messages and automated live weather API strings via Make.com.
- **Insight Extraction:** Isolates meaningful patterns (e.g., critical supply breaks or impending storms) rather than providing generic summaries.
- **Impact Analysis:** Explicitly evaluates real-world consequences, tracking how inventory loss translates to customer disappointment, kitchen chaos, and projected revenue loss.
- **Action Generation:** Programmatically devises a clear, logical cascade of actions (e.g., Hide primary item → Promote high-margin alternative → Adjust combo prices).
- **Action Simulation (Critical Requirement):** Directly triggers live backend adjustments by updating database values, executing automated mock API tasks, and updating communication channels instantly.
- **Outcome Visualization:** Visually reflects the "Before vs. After" state change cleanly on a real-time mobile interface, backed by transparent execution logs and immediate business ROI tracking.

3. The 3-Layer Zero-Code Stack Architecture

The system relies on a modular, decoupled stack designed to maximize robustness and prevent execution latency under strict hackathon constraints:

Layer	Free Visual Tool	Functional Responsibility
Brain	Google Antigravity	Acts as the central reasoning engine. Processes raw text, checks for underlying contradictions, scores business impact, verifies ethical boundaries, and outputs clean, structured JSON payloads. Called exactly once per event.
Bridge	Make.com (Free Tier)	Orchestrates input ingestion, parses Antigravity's JSON, applies numeric/date validation thresholds, routes workflows based on confidence thresholds, and handles secondary executions (Slack, EmailJS).
Body	Supabase + FlutterFlow	Supabase: Houses live menu items, agent trace history, and approval queues. FlutterFlow: Renders a responsive mobile application with real-time operational metrics and an active approval gate.

4. Google Antigravity Integration & Credit Mitigation

Google Antigravity serves as the core orchestration brain. To guarantee the solution fits within the strict **50-call hackathon credit limit**, MenuMind employs a strict **Single-Pass Architecture**:

- All algorithmic filters, conditional branches (e.g., `if price_change > 20%`), basic date math, and string concatenation are handled off-brain via Make.com or Supabase formula fields. This keeps Antigravity calls to exactly 0 for non-semantic logic.
- **The "Dev Mode" Router:** To facilitate hundreds of frontend and database tests without exhausting tokens, a conditional switch is built into Make.com. If the incoming payload contains the keyword `TEST_MODE`, the system bypasses Antigravity entirely and feeds a static, well-formed mock JSON structure to the downstream database layers, enabling free end-to-end frontend refinement.

5. The Master System Prompt

This structured prompt is loaded into the Google Antigravity system instruction field to ensure deterministic, structured reasoning paths:

You are MenuMind, an autonomous cafe operations agent. Your task is to transform unstructured business signals into a structured action plan. You must NOT summarize. You must REASON and ACT.

INPUT FORMAT:

You will receive a text block containing 1-3 signals: Supplier messages (English/Roman Urdu), Weather alerts, or Competitor price updates.

REASONING CHAIN (Log each step in 'agent_trace'):

1. PARSER: Extract item_name, issue_type, severity, and time_sensitivity.
2. CONTRADICTION CHECK: If multiple signals conflict, trust the most recent/specific source. Explain weighting in one sentence.
3. IMPACT SCORER: Rate business impact from 1-10 considering revenue loss, customer disappointment, and kitchen chaos.
4. ETHICAL GUARD: If signals contain crisis keywords ("flood", "disaster", "strike"), REFUSE surge pricing. Recommend cost absorption or portion reduction.
5. CASCADE PLANNER: Generate one primary action and up to two coordinated secondary actions.
6. CONFIDENCE: Assign score from 0.0 to 1.0. If below 0.70, set requires_approval: true.

OUTPUT FORMAT:

Return STRICT JSON format only. No markdown formatting, no conversational text.

```
{
  "agent_trace": [
    {"step": "parser", "finding": "..."},
    {"step": "contradiction", "finding": "..."},
    {"step": "impact", "score": 8, "reason": "..."},
    {"step": "ethical", "decision": "..."}
  ],
  "primary_action": {
    "type": "hide_item | price_change | extend_prep | alert_staff | no_action",
    "item_id": "string",
    "new_value": "string_or_number",
    "reason": "string"
  },
  "secondary_actions": [
    {"type": "...", "item_id": "...", "new_value": "...", "reason": "..."}
  ],
  "confidence": 0.94,
  "requires_approval": false,
  "revenue_impact_pct": -12,
  "customer_message": "..."
}
```

6. Detailed 3-Day Build & Execution Plan

The blueprint is specifically structured to achieve full deployment safely within the standard 72-hour hackathon timeline:

Phase / Day	Hours	Core Focus & Tasks	Tools Involved
Day 1: Ingestion & Skeleton	0 - 4	Provision Google Antigravity environment. Draft, lock, and isolate the Master System Prompt. Set up local workspace.	Antigravity
	4 - 8	Initialize Supabase. Build 3 core data models: menu_items, agent_logs, and pending_approvals. Populate with 20 structural mock rows.	Supabase
	8 - 12	Construct structural layouts in FlutterFlow (Live Dashboard, Approval Gate, Trace Terminal View).	FlutterFlow, Canva
Day 2: Automation & Logic	12 - 16	Construct Make.com Ingestion Pipeline. Establish webhooks and connect raw input models directly to Antigravity.	Make.com, Webhooks
	16 - 20	Build the Executor Pipeline in Make.com. Set up automatic string cleaning via regex parsers to safely extract and format incoming JSON arrays.	Make.com, Regex
	20 - 24	Bind automated notification tasks: route immediate staff alerts to Slack and map consumer notice workflows via EmailJS.	Slack, EmailJS
Day 3: Validation & Demo	24 - 28	Map FlutterFlow fields to active Supabase records. Configure the Typewriter text effect for streaming reasoning data in the Trace view.	FlutterFlow, Supabase
	28 - 32	Execute 5 end-to-end integration scenarios. Perform defensive self-healing pipeline drills (e.g., breaking API URLs to test error catches).	All Systems
	32 - 36	Record a clear 4-minute split-screen walkthrough video using Loom. Package and push standard documentation and schemas to GitHub.	Loom, GitHub

7. Complete Free Tool Stack Limits & Integration Details

Tool Category	Selected Vendor	Free Tier Constraints	Integration Method
AI Brain	Google Antigravity	Hackathon Allocation	Inbound HTTP POST / Outbound structured JSON
Live Database	Supabase	500 MB Storage	Native REST API & FlutterFlow Direct Integration
Automation Core	Make.com	1,000 operations / month	Webhook Triggers & Visual Routing Arrays
UI Application	FlutterFlow	Free / Student Tier	Visual layout canvas with live database sync
Staff Alerts	Slack	Free Workspace	Incoming Slack Webhook Nodes via Make.com
External Signals	OpenWeatherMap	1,000 API calls / day	Make.com HTTP GET Request Modules

8. Innovation & UI "Wow Factors"

1. Real-Time Agent Trace Terminal: Instead of hiding the AI's internal process, MenuMind features an aesthetic terminal screen in FlutterFlow. It parses the `agent_trace` array and uses a typewriter effect to show the step-by-step reasoning (e.g., *"Checking contradictions... none found. Checking ethical guardrails... alert detected, surge pricing blocked."*). This directly guarantees a high score on the Agentic Reasoning evaluation rubric.

2. Human-in-the-Loop Guardrail Gate: High-risk modifications (e.g., price swings exceeding 20%) are held in an isolated approval queue inside Supabase. Managers receive a push notice and can swipe to approve or reject the action inside the mobile app, providing complete human oversight for sensitive business decisions.

3. Instant Financial ROI Indicators: When an automated system update goes live, the manager's dashboard displays a high-visibility badge: **⚡ Saved: \$120 in lost refunds & 45 min of labor**. This shifts the project from a tech concept to a highly practical, commercially valuable asset.

9. Demo Video Storyboard (4-Minute Breakdown)

Constructed to hit every specific evaluation metric within the requested 3-to-5 minute window:

Timestamp	Active Screen Setup	Narrative & Action Sequence	Rubric Target
0:00 - 0:45	Split Screen: Input Form + Kitchen Chaos Graphic	Establish the operational challenge. Paste a chaotic, raw Roman Urdu supplier text message: <i>"Aziiz bhai, gari kharab ho gayi hai, aaj chicken nahi pohnche ga."</i>	Content Ingestion
0:45 - 1:30	Mobile App: Agent Trace Terminal View	Show the typewriter effect outputting the reasoning logs. Highlight the agent parsing the language, identifying the supply block, and calculating the commercial outcome.	Agentic Reasoning (20%)
1:30 - 2:15	Mobile App Dashboard Live Menu Screen	Demonstrate the automated system shift. Show the "Chicken Wrap" instantly flipping to "Unavailable" while the high-margin "Beef Wrap" moves to the top recommendation spot.	Action Simulation (15%)
2:15 - 3:00	Mobile App: Human-in-the-Loop Panel	Simulate a crisis signal (e.g., localized flooding). Show Antigravity triggering the Ethical Guard, blocking automatic surge pricing, and routing a swipe-card approval request to the manager instead.	Edge-Case Handling & Ethics
3:00 - 4:00	Make.com Workspace View & Summary	Display the clean, visual execution pipeline. Walk through the Single-Pass design structure that preserves credit limits, and showcase the fallback mechanisms. Fade to black.	Technical Polish & Architecture

10. Operational Survival Rules & Submission Checklist

Non-Technical Team Strategy Rules:

- **Deterministic JSON Safeguard:** If Antigravity appends conversational text to the payload, place the instruction `"Output strict JSON only. No markdown formatting."` at the very top of your prompt block.
- **Operation Conservation:** Testing end-to-end logic inside Make.com consumes 3 distinct operations per run. Use the manual **"Run Once"** execution toggle with direct text injections during development instead of leaving active schedules on.
- **UI Layout Fixes:** Avoid spending limited hours customizing complex CSS tables inside FlutterFlow. Utilize the native "Admin Dashboard" card components and replace placeholder elements with text objects.

Final Checklist Before Submission:

1. Verify Google Antigravity workspace is initialized and the system prompt is completely locked.
2. Ensure Supabase contains 3 active, accessible tables populated with a minimum of 10 test rows.
3. Confirm Make.com handles incoming payloads, cleans formatting irregularities, and pushes values smoothly.

4. Confirm the FlutterFlow mobile interface displays active state modifications and prints step logs clearly.
5. Validate that the final demo video fits inside the mandatory 3-to-5 minute window and excludes any sensitive personal data.